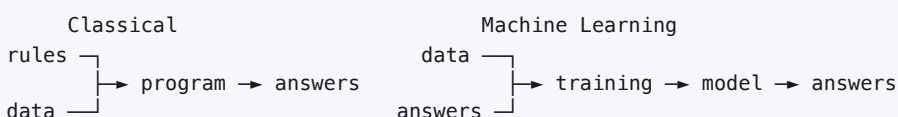


Core Concepts — Machine Learning Foundations

Part 01 —  Machine Learning Foundations · 01-ml-foundations/notes/01-core-concepts.md

1. What machine learning actually is

Classical programming: you write **rules**, feed **data**, get **answers**. Machine learning: you feed **data** + **answers**, and get **rules** (a model) back.



A model is a **parameterised function** $f_{\theta}(x) \rightarrow \hat{y}$ plus a **procedure for choosing θ** that minimises a **loss** on data. That's it. Everything else — deep nets, transformers, LLMs — is a choice of f , a choice of loss, and a lot of engineering around the word "minimise".

When NOT to use ML

- The rule is known and stable (tax brackets $\rightarrow$ write an `if`).
- You cannot get labels, or labels cost more than the problem is worth.
- Errors are unacceptable and unexplainable (safety interlocks).
- You have $< \sim 1,000$ examples and no pretrained model to lean on.
- The data distribution changes faster than you can retrain.

Rule of thumb: ML earns its keep when the pattern is real, repeated, tolerant of error, and too complicated or too fast-changing to hand-code.

2. The three (and a half) learning paradigms

Paradigm	You have	You learn	Canonical examples
Supervised	(x, y) pairs	a mapping $x \rightarrow y$	fraud detection, churn, pricing, image classification
Unsupervised	x only	structure	clustering, topic models, anomaly detection, PCA
Reinforcement	environment + reward	a policy $\text{state} \rightarrow \text{action}$	robotics, game play, RLHF for LLMs
<i>Self-supervised</i>	x only, invent labels from it	representations	word2vec, BERT masking, GPT next-token prediction

Self-supervised learning is the half — and it is the single most important idea for the rest of this track. **Next-token prediction is supervised learning where the labels were free.** That is why LLMs scaled: the internet is an infinite labelled dataset if your label is "the next token".

3. Supervised learning, precisely

Given a dataset $D = \{(x_i, y_i)\}^n$ drawn i.i.d. from unknown distribution $P(X, Y)$, choose θ to minimise **empirical risk**:

$$J(\theta) = \underbrace{(1/n) \sum L(f_{\theta}(x_i), y_i)}_{\text{fit the data}} + \underbrace{\lambda \cdot R(\theta)}_{\text{don't get carried away}}$$

What we actually *want* is low **true risk** $E_{(x,y) \sim P} [L(f_{\theta}(x), y)]$. The whole discipline of generalization is about the gap between those two lines.

Loss functions you should recognise on sight

Task	Loss	Formula	Notes
Regression	MSE	$(y - \hat{y})^2$	punishes outliers hard; assumes Gaussian noise
Regression	MAE	$ y - \hat{y} $	robust to outliers; non-smooth at 0
Regression	Huber	quadratic near 0, linear far	the practical compromise
Binary class.	Binary cross-entropy	$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$	needs calibrated probabilities
Multi-class	Categorical cross-entropy	$-\sum y_k \log \hat{y}_k$	this is the LLM training loss too
Ranking	Pairwise hinge / InfoNCE	margins over pairs	reappears in RAG rerankers & contrastive embeddings

Connect forward: the cross-entropy row is literally the loss used to pretrain GPT-class models (Module 04). Learn it once here, and Module 04 costs you nothing.

4. The model families that matter

4.1 Linear & logistic regression

$\hat{y} = w^T x + b$. Logistic adds a sigmoid: $\hat{y} = \sigma(w^T x + b)$, $\sigma(z) = 1/(1+e^{-z})$. **Why still learn it:** interpretable coefficients, fast, a legitimate baseline, and it is exactly one neuron — the atom of Module 02.

4.2 k-Nearest Neighbours

No training; predict by voting among the k closest examples. Slow at inference, sensitive to scaling and to the curse of dimensionality. **Why care:** this is vector search. Module 08's RAG retrieval is k-NN with embeddings as features.

4.3 Decision trees → Random Forest → Gradient boosting

- A tree splits the feature space by asking greedy yes/no questions that maximise purity (Gini / entropy).
- **Random Forest:** many deep trees on bootstrapped samples + random feature subsets, averaged. Reduces *variance*.
- **Gradient boosting (XGBoost / LightGBM / CatBoost):** trees fit sequentially to the *residuals* of the ensemble so far. Reduces *bias*. Usually the strongest tabular model available.

Still true in 2026: on medium-sized heterogeneous tabular data, gradient-boosted trees beat deep networks in accuracy, training cost, and tuning effort. Don't reach for a neural net because it's fashionable.

4.4 SVMs, Naive Bayes, PCA, k-Means

Know what they do and when they appear: SVM (max-margin, kernels), Naive Bayes (absurdly strong text baseline), PCA (linear dimensionality reduction — the ancestor of embeddings), k-Means (the clustering default).

5. Features: where projects are actually won

Model choice is worth maybe 10% of your outcome. Features and data quality are worth the rest.

- **Numeric:** scale (standardise/min-max) whenever the algorithm is distance- or gradient-based. Trees don't care.
- **Categorical:** one-hot (low cardinality), target/mean encoding (high cardinality — *fit inside CV folds only*), embeddings (very high cardinality).
- **Temporal:** never compute a feature using information from after the prediction timestamp. This is the #1 source of leakage.
- **Text:** bag-of-words → TF-IDF → embeddings (Module 03).
- **Missing values:** missingness is often itself a signal. Add an `is_missing` indicator instead of silently imputing.

Data leakage — the career-defining bug

Leakage is any information in training that will not exist at prediction time. Symptoms: suspiciously high validation scores, a feature with impossible importance, production performance collapse.

Classic leaks: - Scaling/imputing/encoding **before** splitting (the test set's statistics leak into training). - Random splits on time-series data. - IDs, timestamps, or downstream artefacts ("account_closed_reason" predicting churn). - Duplicated rows spanning train and test.

6. Evaluation: choose the metric that matches the cost of being wrong

	Predicted +	Predicted -
Actual +	TP	FN
Actual -	FP	TN

- **Precision** $TP / (TP + FP)$ — of the alarms I raised, how many were real? *Use when false positives are expensive.*
- **Recall** $TP / (TP + FN)$ — of the real cases, how many did I catch? *Use when misses are expensive (cancer, fraud).*
- **F1** — harmonic mean; a compromise, not a goal.
- **ROC-AUC** — ranking quality across thresholds; **misleading under heavy class imbalance.**
- **PR-AUC** — the honest choice when positives are rare.
- **Calibration** — do predicted probabilities mean what they say? Plot a reliability diagram. Required whenever a human or a downstream system consumes the probability (pricing, triage, risk).
- Regression: RMSE (same units, outlier-sensitive), MAE, MAPE (breaks near zero), R^2 .

Accuracy is a trap. In a 99.9%-negative fraud dataset, "always say no" scores 99.9%.

7. The workflow you should internalise

1. Frame → what decision changes because of this prediction?
2. Baseline → majority class / last value / simple rule. Write down its score.
3. Split → train / validation / test, respecting time and groups.
4. Explore → distributions, missingness, target leakage hunt.
5. Features → inside a pipeline so transforms are fit on train folds only.
6. Model → linear → trees → boosting → (maybe) deep.
7. Tune → cross-validated search on the validation set only.
8. Interpret → feature importance, SHAP, partial dependence, error analysis by segment.
9. Test once → touch the test set a single time. More than once and it becomes a validation set.
10. Ship & watch → drift, latency, cost, feedback loops.

Steps 1, 4 and 10 are where seniority shows. Step 6 is the one everyone rushes to.

8. Error analysis (the highest-leverage habit)

Don't stare at aggregate metrics. Take the 100 worst predictions and read them. Tag failures by cause: label noise, missing feature, distribution shift, ambiguous example, genuine model limitation. Fix the top category. Repeat.

This habit transfers *directly* to LLM evals in Module 15 — where it's called "failure taxonomy" and is the difference between a real eval suite and a vanity benchmark.

9. Forward links

Idea here	Where it returns
Cross-entropy loss	Module 04 — LLM pretraining objective
One neuron ($\sigma(w^T x + b)$)	Module 02 — stack them, add backprop
k-NN	Module 08 — vector retrieval
Data leakage	Module 15 — benchmark contamination in LLM evals
Calibration	Module 15 — confidence & abstention in LLM systems
Error analysis	Module 15 — failure taxonomies
Feature importance / SHAP	Module 12 — explainability obligations under governance